

Community Experience Distilled

Practical Maya Programming with Python

Unleash the power of Python in Maya and unlock your creativity

Robert Galanakis

[PACKT] PUBLISHING

Practical Maya Programming with Python

Unleash the power of Python in Maya and unlock your creativity

Robert Galanakis

[PACKT] PUBLISHING

BIRMINGHAM - MUMBAI

Practical Maya Programming with Python

Copyright © 2014 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: July 2014

Production reference: 1180714

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-84969-472-8

www.packtpub.com

Cover image by Andrei Cosmin Cristea (Andrei@undoz.com)

Credits

Author
Robert Galanakis

Reviewers
Harry Boltz
Brian Escribano
Michael Tsai

Commissioning Editor
Akram Hussain

Acquisition Editor
Subho Gupta

Content Development Editor
Dayan Hyames

Technical Editors
Krishnaveni Haridas
Manal Pednekar

Copy Editors
Aditya Nair
Stuti Srivastava

Project Coordinator
Leena Purkait

Proofreaders
Stephen Copestake
Maria Gould
Paul Hindle

Indexers
Mariammal Chettiyar
Tejal Soni
Priya Subramani

Graphics
Sheetal Aute
Yuvraj Mannari
Abhinash Sahu

Production Coordinator
Pooja Chiplunkar

Cover Work
Pooja Chiplunkar

About the Author

Robert Galanakis is a technical artist cum programmer who has worked in various areas of game development. He is the Technical Director of EVE Online at CCP Games, Iceland, where he focuses on Python, Lean, and Agile training and evangelism. In 2008, Rob founded `tech-artists.org`, which is the largest and the most active community focused on tech art on the Internet. He has spoken at Game Developers Conference several times and has also written many articles about tools, pipelines, and culture. His blog can be found at `www.robg3d.com`. He lives in Reykjavík, Iceland, with his wife Casady and their son Marcus.

About the Reviewers

Brian Escribano has over 11 years of experience working in the fields of education, TV, and games. He builds world-class character rigs and animation pipelines for companies such as Nickelodeon, Mirada, and Spark Unlimited. With his deep scripting knowledge in Python and MEL, Brian brings a wealth of expertise and experience to any team he works with.

Michael Tsai attended the Academy of Art University at San Francisco to study Visual Effects. After college, he worked on *Fantastic Four 2 – Rise of the Silver Surfer*, *Red Cliff 2 – The Battle of Red Cliff*, and the stereoscopic version of G-Force. In 2012, Michael received his Master of Entertainment Technology degree (MET) from the Entertainment Technology Center of Carnegie Mellon University. *Elysium* was another feature film he worked on before he joined Schell Games in Pittsburgh as a full-time game artist.

www.PacktPub.com

Support files, eBooks, discount offers and more

You might want to visit `www.PacktPub.com` for support files and downloads related to your book.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at `www.PacktPub.com` and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at `service@packtpub.com` for more details.

At `www.PacktPub.com`, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.



`http://PacktLib.PacktPub.com`

Do you need instant solutions to your IT questions? PacktLib is Packt's online digital book library. Here, you can access, read and search across Packt's entire library of books.

Why Subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print and bookmark content
- On demand and accessible via web browser

Free Access for Packt account holders

If you have an account with Packt at `www.PacktPub.com`, you can use this to access PacktLib today and view nine entirely free books. Simply use your login credentials for immediate access.

Table of Contents

Preface **1**

Chapter 1: Introspecting Maya, Python, and PyMEL **9**

Creating your library **9**

Using the interpreter 10

Finding a place for our library 10

Choosing a development root 11

Creating a function in your IDE 12

Reloading code changes 13

Exploring Maya and PyMEL **13**

Creating an introspection function 15

Understanding Python and MEL types 16

Using the method resolution order 18

PyNodes all the way down 19

Understanding PyMEL data and math types 21

Leveraging the REPL 23

Building the pmhelp function **24**

Creating a query string for a PyMEL object 25

Creating more tests 27

Adding support for modules 29

Adding support for types 30

Adding support for methods 30

Adding support for functions 33

Adding support for non-PyMEL objects 34

Designing with EAFP versus LBYL 37

Code is never complete 38

Opening help in a web browser 38

Summary **40**

Chapter 2: Writing Composable Code 41
Defining composability 41
Identifying anti-patterns of composability 42
Avoiding the use of Boolean flags 44
Evolving legacy code into composable code 45
Rewriting code for composability 46
Getting the first item in a sequence 46
Writing head and tail functions 48
Learning to use list comprehensions 48
Implementing is_exact_type 50
Saying goodbye to map and filter 51
Writing a skeleton converter library 51
Writing the docstring and pseudocode 52
Understanding docstrings and reStructured Text 53
Writing the first implementation 53
Breaking the first implementation 54
Understanding interface contracts 55
Extracting the safe_setparent utility function 56
Learning how to refactor 57
Simplifying the node to joint conversion 58
Learning how to use closures 60
Dealing with node connections 61
Dealing with namespaces 61
Wrapping up the skeleton converter 62
Writing a character creator 63
Stubbing out the character creator 64
Implementing convert_hierarchies_main 65
Implementing convert_hierarchies 66
Decomposing into composable functions 66
Implementing convert_hierarchy 68
Supporting inevitable modifications 69
Improving the performance of PyMEL 72
Defining performance 73
Refactoring for performance 73
Rewriting inner loops to use maya.cmds 75
Summary 75
Chapter 3: Dealing with Errors 77
Understanding exceptions 77
Introducing exception types 78
Explaining try/catch/finally flow control 79

Explaining traceback objects 81
Explaining the exc_info tuple 82
Living with unhandled exceptions 83
Handling exceptions at the application level 83
Golden rules of error handling 84
Focus on the critical path 85
Keep the end user in mind 85
Only catch errors you can handle 86
Avoid partial mutations 87
Practical error handling in Maya 88
Dealing with expensive and mutable state 88
Leveraging undo blocks 90
Dealing with Maya's poor exception design 91
Leveraging the Maya application 92
Dealing with the Maya application 92
Leveraging Python, which is better than MEL 92
Building a high-level error handler 92
Understanding sys.excepthook 93
Using sys.excepthook in Maya 94
Creating an error handler 95
Improving the error handler 96
Inspecting Python code objects 97
Adding filtering based on filename 98
Assembling the contents of an error e-mail 100
Sending the error e-mail 103
Installing the error handler 104
Obeying the What If Two Programs Did This rule 105
Improving the error handler 106
Adding a user interface 106
Using a background thread to send the e-mail 107
Moving beyond e-mail 107
Capturing locals 107
Attaching log files 108
Summary 109

Chapter 4: Leveraging Context Managers and Decorators in Maya 111

Inverting the subroutine 111
Introducing decorators 113
Explaining decorators 113
Wrapping an exporter with a decorator 117

Introducing context managers **118**
Writing the undo_chunk context manager 121
Writing the undo_on_error context manager 122
Contrasting decorators and context managers 123
Context managers for changing scene state **124**
Building the set_file_prompt context manager 125
Building the at_time context manager 126
Building the with_unit context manager 126
Building the set_renderlayer_active context manager 127
Building the set_namespace_active context manager 127
Improving on future versions of Maya 129
Creating the denormalized_skin context manager **129**
Safely swapping vertex influences 129
Addressing performance concerns 131
Creating a decorator to record metrics **133**
Getting a unique key 134
Recording duration 134
Reporting duration 135
Handling errors 136
Advanced decorator topics **138**
Defining decorators with arguments 138
Decorating PyMEL attributes and methods 139
Stacking decorators 139
Using Python's decorator library 140
Doing decorators the right way 140
Summary **141**
Chapter 5: Building Graphical User Interfaces for Maya **143**
Introducing Qt, PyQt, and PySide **143**
Introducing Qt widgets 144
Introducing Qt layouts 145
Understanding Qt main windows and sorting 145
Introducing Qt signals 146
Establishing rules for crafting a GUI **146**
Prefer pure PySide GUIs where possible 146
Use command-style UI building where necessary 146
Avoid the use of .ui files 147
Installing PySide **147**
Supporting PySide and PyQt **148**
Creating the hierarchy converter GUI **149**
Creating the window 149

Running a Python file as a script 150
Introducing the QApplication class 151
Understanding the event loop 151
Running your GUI 152
Designing and building your GUI 153
Defining control, container, and window widgets 153
Adding the rest of the widgets 155
Hooking up the application to be effected by the GUI 156
Hooking up the GUI to be effected by the application 158
Simulating application events 160
Considering alternative implementations 161
Integrating the tool GUI with Maya 162
Opening the tool GUI from Maya 162
Getting the main Maya window as a QMainWindow 163
Making a Qt window the child of Maya's window 164
Using Python's reload function with GUIs 165
Emitting a signal from Maya 166
Connecting Maya to a signal 167
Verifying the hierarchy converter works 169
Working with menus 169
Creating a top-level menu 169
Getting the Qt object from a Maya path 170
Changing the font of a widget 171
Marking menus as new 172
Creating a test case 173
Adding a persistence registry 174
Verifying the new menu marker works 176
Using alternative methods to style widgets 176
Working with Maya shelves 177
Summary 178
Chapter 6: Automating Maya from the Outside 179
Controlling Maya through request-reply 180
Using a Python client and Maya server 180
Controlling Python through exec and eval 180
Handling problems with IPC 181
Installing ZeroMQ 181
Demonstrating request-reply with ZeroMQ 182
Explaining connection strings, ports, bind, and connect 183
Designing the automation system 184
Pairing one client and one server 184

Bootstrapping the server from the client 185
The client-server handshake 185
Defining the server loop 188
Serializing requests and responses 188
Choosing what the server does 189
Handling exceptions between client and server 189
Understanding the Maya startup routine 191
Using batch mode versus GUI mode 192
Choosing a startup configuration mechanism 192
Using command line options 192
Using environment variables 193
Building the request-reply automation system 193
Creating a Python package 194
Launching Maya from Python 194
Automatically killing the server 196
Creating a basic Maya server 197
Running code at Maya startup 198
Understanding eval and exec 199
Adding support for eval and exec 201
Adding support for exception handling 202
Adding support for timeouts 206
Adding support for the client-server handshake 208
Practical uses and improvements 211
Batch processing using Maya 211
Running a server in a Maya GUI session 213
Running automated tests in Maya 214
Adding support for logging 214
Supporting multiple languages and applications 215
Supporting control from a remote computer 215
Designing an object-oriented system 216
Evaluating other RPC frameworks 216
Summary 216
Chapter 7: Taming the Maya API 217
Explaining types 218
Dicts all the way down 218
Using custom types to simplify code 220
Introducing inheritance by drawing shapes 221
Introducing Maya's API and architecture 225
Understanding the OpenMaya bindings 226
Navigating the Maya API Reference 227

Understanding MObjects and function sets 229
Learning the Maya Python API by example 230
Converting a name to an MObject node 230
Getting the name of an MObject 231
Getting the hash of a node 231
Building a mesh 232
Setting mesh normals 238
Using MScriptUtil to call a method 241
Using OpenMaya for callbacks 243
Comparing Maya Python API and PyMEL 246
Creating a Maya Python plugin 247
The life of a Python plugin 248
Creating the sound player library 249
Creating the plugin file 250
Reloading plugins 252
Adding a command flag 252
Comparing the OpenMaya and scripting solutions 255
Using PyMEL in a plugin that loads during startup 256
Summary 257
Chapter 8: Unleashing the Maya API through Python 259
Understanding Dependency Graph plugins 260
Building a simple node plugin 260
Understanding plugin type IDs 262
Defining inputs, outputs, and the initializer 263
Creating the compute method 266
Taming the non-Pythonic Maya API 268
Demystifying Python metaprogramming 268
Rethinking type creation 269
Exploring the type function 270
The importance of being declarative 271
Designing the node factory 273
Designing plugin nodes 273
Designing the attribute specification 274
Designing the node type specification 275
Building the node factory 276
Specifying attributes 276
Creating attributes 277
Specifying a node 279
Using partial application to create attributes 281
Creating a node 282

Slaying the compute method 285
Extending the node factory 289
Supporting string and color attributes 289
Supporting enum attributes 290
Supporting transform nodes 292
Overriding MPxNode methods 293
Summary 294
Chapter 9: Becoming a Part of the Python Community 295
Understanding Open Source Software 296
Differentiating OSS from script download sites 296
Defining what a third-party module is 297
Creating a site directory for third-party modules 298
Explaining the site directory 298
Creating a new site directory for Maya 299
Establishing the site directory at startup 299
Working with Python distributions in Maya 300
Using the Python Package Index 300
Adding a source distribution to Maya 300
Adding an egg or wheel to Maya 301
Using binary distributions on Windows 302
Using pip to install third-party modules 303
Contributing to the open source community 303
Designing Maya Python code for open source 304
Starting an open source project 306
Distributing your project 307
Engaging with the wider community 308
Summary 309
Appendix: Python Best Practices 311
The args and kwargs parameters 311
String formatting 313
String concatenation 315
Raw strings and string literals 316
Path building and manipulation 317
Unicode strings 318
Using the doctest module 319
Adopting Test-Driven Development 320
Using the GitHub repository for this book 321
Index 323

Preface

When Autodesk added support for Python into Maya 8.5, few people understood the implications. It was a decision that has fundamentally changed the way 3D art gets done. Now, years later, we stand on the edge of realizing its promise.

The promise of Python in Maya goes beyond just a familiar language with a great syntax. Any language could have been chosen to bind to Maya; and most would have been more familiar, and with a better syntax than MEL, and easier to use than C++. So, why Python?

The promise goes beyond a powerful language with lots of built-in features. Python is said to have *batteries included*, but so do other languages, and Autodesk certainly has lots of batteries in Maya that now also exist in Python. So, again, why Python?

The promise goes beyond having a single language for scripting, API use, and plugins. It goes beyond the endless third-party libraries maintained by a large community. It goes beyond having powerful development tools.

The promise of Python in Maya is all of these things and more. You can learn how to use the language by leveraging a wide variety of resources that have nothing to do with Maya. You can easily translate what you know of MEL and the C++ API and use it in Python, but with an improved development velocity and maintainability of code. You can use your favorite standard Python editor and tools. You can learn about the language from a technical and design perspective and apply that to improve your programming in Maya. You can be part of a large, vibrant, diverse community of developers on the forefront of multiple areas of technology.

Join me as we explore topics that will allow you to unleash the power of Maya through Python. Together, we'll learn how Python works both under the hood and over it, how Maya integrates with Python, and how the elegant PyMEL builds on that integration. We will drill down into what makes Python code beautiful and idiomatic, and how we can use these concepts and Python's language features to make our Maya Python code expressive and elegant. We will leverage third-party solutions for networking and user interfaces, to compliment and extend what is included with Maya and Python. We will decouple Python code from Maya dependencies, making our work go smoother and faster

This book is not a reference. It is not a cookbook, and it is not a comprehensive guide to Maya's Python API. It is a book that will teach you how to write better Python code for use inside of Maya. It will unearth interesting ways of using Maya and Python to create amazing things that wouldn't be possible otherwise. While there is plenty of code in this book that I encourage you to copy and adapt, this book is not about providing recipes. It is a book to teach skills and enable.

This is a book which, I hope, helps realize the promise of Python in Maya.

What this book covers

Chapter 1, *Introspecting Maya, Python, and PyMEL*, explores how Maya and Python work individually and together to create a powerful programming and scripting environment. It covers some of the key technical underpinnings for the rest of the book.

Chapter 2, *Writing Composable Code*, introduces the practice of writing code that can be reused in many places. Composable code is a fundamental concept for the rest of the skills taught in this book.

Chapter 3, *Dealing with Errors*, teaches you all about exceptions and errors in Maya and Python. We explore several strategies for handling them effectively.

Chapter 4, *Leveraging Context Managers and Decorators in Maya*, covers context managers and decorators, which are two powerful features of Python, and how they can be used to simplify your code.

Chapter 5, *Building Graphical User Interfaces for Maya*, demonstrates the PySide and PyQt frameworks, how to abstract your user interface code from underlying logic, and a strategy of building GUIs to maximize maintainability and productivity.

Chapter 6, *Automating Maya from the Outside*, shows how Maya can be controlled from another process, explains how request-reply systems work, and builds a fully featured automation system on these principles.

Chapter 7, *Taming the Maya API*, introduces the Maya Python API and how types and classes work in Python and Maya. It contains a number of examples to demonstrate the API, as well as a Maya command plugin.

Chapter 8, *Unleashing the Maya API through Python*, covers the creation of a library to easily create Maya plugin nodes, demonstrating how to map the Maya API onto Pythonic idioms using metaprogramming.

Chapter 9, *Becoming a Part of the Python Community*, goes over the concepts behind open source software, demonstrates how to find and use third-party modules, explains how to create your own open source project, and tours the Python and Maya programming communities.

Appendix, *Python Best Practices*, explains in detail various Python language features and miscellaneous, but very relevant, topics.

What you need for this book

You will need a copy of Autodesk Maya 2013 for this book. Newer versions are fine, and older versions that use Python 2.6 (2011, 2012) should be acceptable as well. Any operating system capable of running Maya (Windows, OS X, Linux) should work, though you will need to translate things such as file paths to what is appropriate on your system.

I would also suggest having an install of Python 2.6 or 2.7 outside Maya for exploring and running some of the samples that can be run from the interactive interpreter prompt. You can download Python 2.6 or 2.7 from `http://www.python.org/download`, or it may be installed on your Mac or Linux OS already!

Finally, I strongly suggest installing both a powerful text editor and an **Integrated Development Environment** (**IDE**). Python is a real programming language, and you should use the powerful tools available. If you are an experienced Python user already happy with `vim`, I don't expect to convert you. But if you are a converted MEL scripter playing around in Notepad, it is time to embrace your good fortune!

For a text editor, **Sublime Text** (`http://www.sublimetext.com`) is popular, cross-platform, and free to use on an unlimited trial. **Notepad++** (`http://notepad-plus-plus.org`) is excellent if you are on Windows and prefer free and open source. There are dozens of other good text editors, and if the two editors mentioned here do not tickle your fancy, you should keep trying until you find one that does.

Finally, the choice of IDE is usually a contentious topic. For Python, however, I can confidently say **PyCharm** (`http://www.jetbrains.com/pycharm/`) by JetBrains is the premiere IDE, and my personal favorite. It has a free and quite powerful Community Edition as well. Other popular options are **Wing IDE** (`http://www.wingware.com`) and **Eclipse** with **PyDev** (`http://pydev.org`). Experiment with a few different programs, but whatever you do, move past IDLE, the IDE bundled with Python!

Who this book is for

- Do you currently use Python with Maya and ask yourself: "Can I do better?"
- Are you a MEL scripter who has started using Python and want to know what all the fuss is about?
- Are you a Python programmer who is starting with Maya and believes there must be a better way?
- Have you been using Python in Maya for a while but work hard to continuously improve?

Some basic experience with Python and Maya is expected. This book does not cover those most primitive topics that are inevitably learned through introductory Maya and Python use.

Even more than experience, this book requires a willingness to learn. Some of the more advanced or unfamiliar topics may feel a bit like learning how to ride a bicycle, but keep peddling and you'll get the hang of things in no time.

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles, and an explanation of their meaning.

Code words in text are shown as follows: "We can include other modules through the use of the `import` statement."

A block of code is set as follows:

```
def spam():
    return 'spam!'
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
def more_spam():
    spams = ' '.join([spam()] * 5)
    return spams
```

Any command line input or output is written as follows:

```
> mayapy --version
Python 2.6.4
```

Code meant to be entered by the Python interactive interpreter uses its familiar conventions. Input lines are prefixed with ">>>". Continuations for multiline statements are prefixed with ". . .". Output from the interpreter has no prefix:

```
>>> 'This is input'.replace('in', 'out')
'This is output'
>>> if True:
...     print 'Hello!'
Hello
```

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "Clicking on the **Next** button moves you to the next screen".



Warnings or important notes appear in a box like this.



Tips and tricks appear like this.

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book—what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an e-mail to `feedback@packtpub.com`, and mention the book title via the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide on `www.packtpub.com/authors`.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support` and register to have the files e-mailed directly to you.

The code for this book is also available on GitHub, at `https://github.com/rgalanakis/practicalmayapython`. See the *Appendix*, *Python Best Practices,* for more information about the GitHub repository.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you would report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting `http://www.packtpub.com/submit-errata`, selecting your book, clicking on the **errata submission form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded on our website, or added to any list of existing errata, under the Errata section of that title. Any existing errata can be viewed by selecting your title from `http://www.packtpub.com/support`.

Piracy

Piracy of copyright material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at `copyright@packtpub.com` with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at `questions@packtpub.com` if you are having a problem with any aspect of the book, and we will do our best to address it.

You can also contact the author, Robert Galanakis, at `rob.galanakis@gmail.com`.

1
Introspecting Maya, Python, and PyMEL

Maya and Python are both excellent and elegant tools that can together achieve amazing results. And while it may be tempting to dive in and start wielding this power, it is prudent to understand some basic things first. Knowledge of the fundamentals will provide the platform from which we will grow great skills and conquer our obstacles throughout the rest of this book.

In this chapter, we will look at Python as a language, Maya as a program, and PyMEL as a framework. We will begin by briefly going over how to use the standard Python interpreter, the Maya Python interpreter, the Script Editor in Maya, and your **Integrated Development Environment** (**IDE**) or text editor in which you will do the majority of your development. Our goal for the chapter is to build a small library that can easily link us to documentation about Python and PyMEL objects. Building this library will illuminate how Maya, Python and PyMEL are designed, and demonstrate why PyMEL is superior to `maya.cmds`. We will use the powerful technique of **type introspection** to teach us more about Maya's node-based design than any Hypergraph or static documentation can. Along the way we will explore some core concepts that will reoccur throughout later chapters.

Creating your library

There are generally three different modes you will be developing in while programming Python in Maya: using the mayapy interpreter to evaluate short bits of code and explore ideas, using your Integrated Development Environment to work on the bulk of the code, and using Maya's Script Editor to help iterate and test your work. In this section, we'll start learning how to use all three tools to create a very simple library.

Using the interpreter

The first thing we must do is find your mayapy interpreter. It should be next to your Maya executable, named `mayapy` or `mayapy.exe`. It is a Python interpreter that can run Python code as if it were being run in a normal Maya session. When you launch it, it will start up the interpreter in **interactive mode**, which means you enter commands and it gives you results, interactively. The `>>>` and `...` characters in code blocks indicate something you should enter at the interactive prompt; the code listing in the book and your prompt should look basically the same. In later listings, long output lines will be elided with `...` to save on space.



Most of the interactive samples can be run as code through `doctest`. See *Appendix*, *Python Best Practices*, for more information.

Start a `mayapy` process by double clicking or calling it from the command line, and enter the following code:

```
>>> print 'Hello, Maya!'
Hello, Maya!
>>> def hello():
...     return 'Hello, Maya!'
...
>>> hello()
'Hello, Maya!'
```

The first statement prints a string, which shows up under the prompting line. The second statement is a multiline function definition. The `...` indicates the line is part of the preceding line. The blank line following the `...` indicates the end of the function. For brevity, we will leave out empty `...` lines in other code listings. After we define our `hello` function, we invoke it. It returns the string `"Hello, Maya!"`, which is printed out beneath the invocation.

Finding a place for our library

Now, we need to find a place to put our library file. In order for Python to load the file as a module, it needs to be on some path where Python can find it. We can see all available paths by looking at the path list on the `sys` module.

```
>>> import sys
>>> for p in sys.path:
...     print p
C:\Program Files\Autodesk\Maya2013\bin\python26.zip
C:\Program Files\Autodesk\Maya2013\Python\DLLs
C:\Program Files\Autodesk\Maya2013\Python\lib
```

```
C:\Program Files\Autodesk\Maya2013\Python\lib\plat-win
C:\Program Files\Autodesk\Maya2013\Python\lib\lib-tk
C:\Program Files\Autodesk\Maya2013\bin
C:\Program Files\Autodesk\Maya2013\Python
C:\Program Files\Autodesk\Maya2013\Python\lib\site-packages
```

A number of paths will print out; I've replicated what's on my Windows system, but yours will almost definitely be different. Unfortunately, the default paths don't give us a place to put custom code. They are application installation directories, which we should not modify. Instead, we should be doing our coding outside of all the application installation directories. In fact, it's a good practice to avoid editing anything in the application installation directories entirely.

Choosing a development root

Let's decide where we will do our coding. We'll call this location the development root for the rest of the book. To be concise, I'll choose `C:\mayapybook\pylib` to house all of our Python code, but it can be anywhere. You'll need to choose something appropriate if you are on OS X or Linux; we will use `~/mayapybook/pylib` as our path on these systems, but I'll refer only to the Windows path except where more clarity is needed. Create the development root folder, and inside of it create an empty file named `minspect.py`.

Now, we need to get `C:\mayapybook\pylib` onto Python's `sys.path` so it can be imported. The easiest way to do this is to use the `PYTHONPATH` environment variable. From a Windows command line you can run the following to add the path, and ensure it worked:

```
> set PYTHONPATH=%PYTHONPATH%;C:\mayapybook\pylib
> mayapy.exe
>>> import sys
>>> 'C:\\mayapybook\\pylib' in sys.path
True
>>> import minspect
>>> minspect
<module 'minspect' from '...\minspect.py'>
```

The following is the equivalent commands on OS X or Linux:

```
$ export PYTHONPATH=$PYTHONPATH:~/mayapybook/pylib
$ mayapy
>>> import sys
>>> '~/mayapybook/pylib' in sys.path
True
>>> import minspect
>>> minspect
<module 'minspect' from '.../minspect.py'>
```

There are actually a number of ways to get your development root onto Maya's path. The option presented here (using environment variables before starting Maya or mayapy) is just one of the more straightforward choices, and it works for mayapy as well as normal Maya. Calling `sys.path.append('C:\\mayapybook\\pylib')` inside your `userSetup.py` file, for example, would work for Maya but not mayapy (you would need to use `maya.standalone.initialize` to register *user paths*, as we will do later).

Using `set` or `export` to set environment variables only works for the current process and any new children. If you want it to work for unrelated processes, you may need to modify your global or user environment. Each OS is different, so you should refer to your operating system's documentation or a Google search. Some possibilities are `setx` from the Windows command line, editing `/etc/environment` in Linux, or editing `/etc/launchd.conf` on OS X. If you are in a studio environment and don't want to make changes to people's machines, you should consider an alternative such as using a script to launch Maya which will set up the `PYTHONPATH`, instead of launching the `maya` executable directly.

Creating a function in your IDE

Now it is time to use our IDE to do some programming. We'll start by turning the path printing code we wrote at the interactive prompt into a function in our file. Open `C:\mayapybook\pylib\minspect.py` in your IDE and type the following code:

```
import sys
def syspath():
    print 'sys.path:'
    for p in sys.path:
        print '  ' + p
```

Save the file, and bring up your mayapy interpreter. If you've closed down the one from the last session, make sure `C:\mayapybook\pylib` (or whatever you are using as your development root) is present on your `sys.path` or the following code will not work! See the preceding section for making sure your development root is on your `sys.path`.

```
>>> import minspect
>>> reload(minspect)
<module 'minspect' from '...\minspect.py'>
>>> minspect.syspath()
C:\Program Files\Autodesk\Maya2013\bin\python26.zip
C:\Program Files\Autodesk\Maya2013\Python\DLLs
C:\Program Files\Autodesk\Maya2013\Python\lib
```

```
C:\Program Files\Autodesk\Maya2013\Python\lib\plat-win
C:\Program Files\Autodesk\Maya2013\Python\lib\lib-tk
C:\Program Files\Autodesk\Maya2013\bin
C:\Program Files\Autodesk\Maya2013\Python
C:\Program Files\Autodesk\Maya2013\Python\lib\site-packages
```

First, we import the `minspect` module. It may already be imported if this was an old mayapy session. That is fine, as importing an already-imported module is fast in Python and causes no side effects. We then use the `reload` function, which we will explore in the next section, to make sure the most up-to-date code is loaded. Finally, we call the `syspath` function, and its output is printed. Your actual paths will likely vary.

Reloading code changes

It is very common as you develop that you'll make changes to some code and want to immediately try out the changed code without restarting Maya or mayapy. You can do that with Python's built-in `reload` function. The `reload` function takes a module object and reloads it from disk so that the new code will be used.

When we jump between our IDE and the interactive interpreter (or the Maya application) as we did earlier, we will usually reload the code to see the effect of our changes. I will usually write out the `import` and `reload` lines, but occasionally will only mention them in text preceding the code.

Keep in mind that reload is not a magic bullet. When you are dealing with simple data and functions as we are here, it is usually fine. But as you start building class hierarchies, decorators, and other things that have dependencies or state, the situation can quickly get out of control. Always test your code in a fresh version of Maya before declaring it done to be sure it does not have some lingering defect hidden by reloading.

Though once you are a master Pythonista you can ignore these warnings and figure out how to reload just about anything!

Exploring Maya and PyMEL

Now we will start digging into Maya and PyMEL. Let's begin by initializing Maya in the mayapy interpreter so we can use more than just standard Python functionality. We do this by calling `maya.standalone.initialize`, as shown in the following code:

```
>>> import maya.standalone
>>> maya.standalone.initialize()
>>> import pymel.core as pmc
>>> xform, shape = pmc.polySphere()
```

The import of `pymel.core` will implicitly call `maya.standalone.initialize` automatically, but I do it explicitly here so it's clear what's going on. In the future, you can generally skip the call to `maya.standalone.initialize` and just import `pymel.core`.

There is a lot we can discover about these PyMEL objects, which represent Maya nodes, using basic Python. For example, to see the **type** of either of our objects, we can use the built-in `type` function (we will dig much deeper into types later in this chapter).

```
>>> type(xform)
<class 'pymel.core.nodetypes.Transform'>
>>> type(shape)
<class 'pymel.core.nodetypes.PolySphere'>
```

To see the names of all the attributes on our shape, we can use the built-in `dir` function:

```
>>> dir(xform)
['LimitType', 'MAttrClass', ..., 'zeroTransformPivots']
```

We can use the built-in `getattr` function with an object and a name of one of its attributes to get the value of the attribute.

```
>>> getattr(xform, 'getShape')
<bound method Transform.getShape of nt.Transform(u'pSphere1')>
>>> getattr(xform, 'translate')
Attribute(u'pSphere1.translate')
```



Note that we are using the `getattr` built-in Python function, not to be confused with the `maya.cmds.getAttr` function or its equivalent PyMEL version. We will not use the `getAttr` function at all in this book. While using `getAttr` may be more familiar if you are coming from the world of MEL and `maya.cmds`, using `getattr` is familiar if you are coming from the much saner world of Python. We should take full advantage of Python and do things the Python, not MEL, way.

We can combine all of these techniques and use the `inspect` module from Python's standard library (commonly referred to as the `stdlib`) to filter for interesting information about our object.

```
>>> import inspect
>>> methods = []
>>> for a in dir(xform):
...     attr = getattr(xform, a)
...     if inspect.ismethod(attr):
```

```
...         methods.append(attr)
>>> attrs = xform.listAttr()
>>> methods
[<bound method Transform.__add__ of nt.Transform(u'pSphere1')>, ...]
>>> attrs
[Attribute(u'pSphere1.message'), ...]
```

In the preceding code, we use the `dir` function to get every Python attribute from our PyMEL transform instance, and then filter them into a list of methods. We then use the `listAttr` method to get a list of Maya attributes on the transform. Based on this data, we can begin to see how the node is structured. Maya attributes are represented by instances of `Attribute`, and methods are usually helpers.



Using `for` loops and `if` statements that append to a list is not a good practice. We'll replace them with a much nicer syntax called **list comprehensions** in *Chapter 2, Writing Composable Code*.

Creating an introspection function

We'll create a new function in `minspect.py` that will print out interesting information about the object we pass it. Open `C:\mayapybook\pylib\minspect.py` in your IDE or text editor. First, we will add an import to the top of the file (`import sys` should already be there). As a matter of style, imports should always go at the top of the file, one per line, divided into groups. Refer to Python's style guide, called **PEP8**, at `http://www.python.org/dev/peps/pep-0008/#imports`. We will refer back to **PEP8** several times throughout this book.

```
import pymel.core as pmc
import sys
```

Now let's use our earlier techniques and our understanding of Maya to print some information about an object. We can use the `name` method on `PyNode` to provide a simple string representation for a node.

```
def info(obj):
    """Prints information about the object."""

    lines = ['Info for %s' % obj.name(),
             'Attributes:']
    # Get the name of all attributes
    for a in obj.listAttr():
        lines.append('  ' + a.name())
    result = '\n'.join(lines)
    print result
```

You'll notice that instead of repeatedly printing, we put our data into a list and print it only once at the end. We do this for three reasons.

First, appending substrings to a list of strings is faster than incrementally concatenating (adding) them into a large string. Never concatenate many strings together in Python. For example, consider the following expression.

```
'Hello, ' + username + ', it is ' + now + ' right now.'
```

Evaluating this expression creates several unnecessary intermediate strings that are immediately thrown away. Ideally we should minimize transient objects.

Second, joining a list of strings is a well-known pattern. It is said to be more Pythonic than string concatenations.



There is no formal definition for what Pythonic means, but here is an attempt: *something is said to be Pythonic when enough people have said it is Pythonic, or it is very similar to something that is Pythonic*. You should learn to use Pythonic idioms wherever possible, and I will point them out when we run across them.

Finally and most importantly, building a list and printing at the end is more maintainable and easier to test because we are separating decisions (creating the result string) from dependencies (the output stream we print to). If this were production code we'd remove the `print` (dependency) entirely so we can easily test the decisions in `info`. We facilitate that future change by putting `print` into the last line so our dependency is in one place instead of on every line.



Note the use of the `%` operator in the `info` function. We are using **string formatting**. If you are not familiar with string formatting in Python, refer to *Appendix*, *Python Best Practices,* for more information about it.

We can continue adding to this function to express more information about a node. Try adding support for printing relatives by using `obj.listRelatives()` if you are comfortable.

Understanding Python and MEL types

Python's `type` function, which we used earlier, returns the type of an object. Every object in Python, including the `type` object, has a type, and PyMEL is no different.

```
>>> type([])
<type 'list'>
```

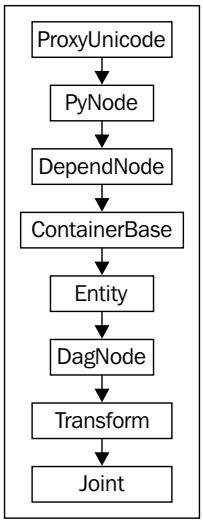
```
>>> type(type([]))
<type 'type'>
>>> type(xform)
<class 'pymel.core.nodetypes.Transform'>
```

You may also be familiar with MEL's *type strings*. Because MEL does not possess a rich type system like Python, *typing* (if it can be called that) is done with strings, as we can see in the following example. The usages of MEL type strings are highlighted.

```
>>> pmc.joint()
nt.Joint(u'joint1')
>>> pmc.polySphere()
[nt.Transform(u'pSphere2'), nt.PolySphere(u'polySphere2')]
>>> pmc.ls(type='joint')
[nt.Joint(u'joint1')]
>>> pmc.ls(type='transform')
[...nt.Joint(u'joint1'), nt.Transform(u'pSphere1'), ...]
>>> pmc.ls(type='shape')
[...nt.Mesh(u'pSphereShape1'), ...]
```

This MEL type, as we'll call it, is very useful while scripting, but not very descriptive. For example, we need to know in advance that a joint is a specific type of transform, and thus returned from invoking `pmc.ls(type='transform')`. This relationship is not clearly expressed.

In contrast, these taxonomic relationships are much better expressed through Python's type system. If we go to the PyMEL documentation for its `Joint` class, we can see the following diagram of its type hierarchy:



This type hierarchy mimics Maya's underlying object oriented architecture, and we can use it to understand things about nodes we may not be totally familiar with. For example, we can know that a `Joint` has `translate/rotate/scale` attributes because it is a subclass of `Transform`. A subclass inherits the behavior, such as the attributes and methods, of its base class. A subclass is commonly called a **child class**, and a base class is commonly called a **parent class** or **superclass**.

Notice in the example below how the `__bases__` attribute indicates that `Transform` is the base class of the `Joint` class.

```
>>> j = pmc.joint()
>>> j.type()
u'joint'
>>> type(j)
<class 'pymel.core.nodetypes.Joint'>
>>> type(j).__bases__
(<class 'pymel.core.nodetypes.Transform'>,)
>>> j.translate, j.rotate
(Attribute(u'joint2.translate'), Attribute(u'joint2.rotate'))
```

We will look into PyMEL's type hierarchies for the next few sections as a means of understanding how PyMEL nodes work. Don't get intimidated if these concepts are new. We won't be creating any of our own types in this section, and if you are familiar with Maya's nodes, the type hierarchies we are going to examine should be rather intuitive. Later in the book, once we are more familiar with Python and PyMEL, several exercises will require creating our own types.

Using the method resolution order

Even more useful than the `__bases__` attribute is the `__mro__` attribute. **Method Resolution Order** (**MRO**) is the order Python visits different types so it can figure out what to actually call. You generally don't need to understand the MRO mechanisms (they can be complex), but looking at the MRO will help you understand all the type information about an object. Let's look at the MRO for the `Joint` type:

```
>>> type(j).__mro__
(<class 'pymel.core.nodetypes.Joint'>,
<class 'pymel.core.nodetypes.Transform'>,
<class 'pymel.core.nodetypes.DagNode'>,
<class 'pymel.core.nodetypes.Entity'>,
<class 'pymel.core.nodetypes.ContainerBase'>,
<class 'pymel.core.nodetypes.DependNode'>,
<class 'pymel.core.general.PyNode'>,
<class 'pymel.util.utilitytypes.ProxyUnicode'>,
<type 'object'>)
```

This makes sense: a joint node is a special type of transform node, which is a type of DAG node, which is a type of dependency node, and so on, mirroring the inheritance diagram we previously saw. The fact that reality meets expectation here is a great testament to Maya and PyMEL.

When a call to `j.name()` is invoked, Python will walk along the MRO looking for the first appropriate method to call. In the case of PyMEL, looking at the MRO often tells us a lot about how an object behaves. This is not always the case, however. Python allows much more dynamic resolution mechanisms. We will not use these mechanisms, such as `__getattr__` or `__getattribute__`, much in this book, but you should be aware that the MRO may not always tell the whole story.

Let's add the collection of type and MRO information into the `minspect.py` file's `info` function:

```
def info(obj):
    """Prints information about the object."""

    lines = ['Info for %s' % obj.name(),
             'Attributes:']
    # Get the name of all attributes
    for a in obj.listAttr():
        lines.append('  ' + a.name())
    lines.append('MEL type: %s' % obj.type())
    lines.append('MRO:')
    lines.extend(['  ' + t.__name__ for t in type(obj).__mro__])
    result = '\n'.join(lines)
    print result
```

PyNodes all the way down

In Python, there's a saying, "Its objects all the way down." This means that everything in Python, including numbers, strings, modules, functions, types, and so on, are all just objects. In MEL and `maya.cmds` I like to say, "Its strings all the way down." Because the type system in MEL and `maya.cmds` is so rudimentary, many things must be handled through strings. And in PyMEL, I like to say, "It's PyNodes all the way down."



The saying is adapted from "Its turtles all the way down." It is left as an exercise to the reader to uncover the origin of this quote. It may also be said in Python that, "Its dicts all the way down." Python is a language of many clever sayings.

Let's look at our PyMEL transform node to better understand how it is "PyNodes all the way down."

```
>>> type(xform).__mro__
(<class 'pymel.core.nodetypes.Transform'>,
<class 'pymel.core.nodetypes.DagNode'>,
<class 'pymel.core.nodetypes.Entity'>,
<class 'pymel.core.nodetypes.ContainerBase'>,
<class 'pymel.core.nodetypes.DependNode'>,
<class 'pymel.core.general.PyNode'>,
<class 'pymel.util.utilitytypes.ProxyUnicode'>,
<type 'object'>)
>>> type(xform.translate).__mro__
(<class 'pymel.core.general.Attribute'>,
<class 'pymel.core.general.PyNode'>,
<class 'pymel.util.utilitytypes.ProxyUnicode'>,
<type 'object'>)
```

There are a number of very interesting things going on in this short listing.

First, our two types—along with all PyMEL types, in fact—inherit from both `PyNode` and `ProxyUnicode` (as well as `object`, which all Python types inherit from). The `PyNode` type represents any Maya node (DAG/dependency nodes, attributes, Maya windows, and so on). Vitally, `Attributes` are also `PyNodes`. If we look at the PyMEL help for `PyNode`, we can see the distinguishing features of `PyNodes` are that they have a name/identity, connections, and history.

Second, anything that inherits from `DependNode` has attributes. So predictably, `Attributes` are not a `DependNode`, but our `Transform` is.

Third, `Transform` is also a `DagNode`. We can use our knowledge of Maya (or graph theory, if you're into that) to infer that this means the object can have a parent, children, instances, and so on. This is a great example where our knowledge of Maya maps directly onto PyMEL, and we aren't required to learn a new paradigm to understand how the PyMEL framework works.



We build custom Maya nodes in *Chapter 7, Taming the Maya API.*

Finally, if we look at the `__mro__` for `a Joint`, we will see the following information:

```
>>> type(pmc.joint()).__mro__
(<class 'pymel.core.nodetypes.Joint'>,
<class 'pymel.core.nodetypes.Transform'>,
...,
<type 'object'>)
```

We can immediately understand much of how a PyMEL `Joint` works if we already understand `Transform`. In fact, everything about `Transform` is also true for every `Joint`. The deduction that every `Joint` behaves like a `Transform` is known as the **Liskov substitution principle**. It states, roughly, that *if S is a subclass of T, then a program can use an instance of S instead of T without a change in behavior*. It is a fundamental principle of good object-oriented design and manifests itself in well-designed frameworks such as PyMEL.

The fact that types inherit the behavior of their parents is important to keep in mind as you go through the rest of this book and program with PyMEL. Don't worry if you don't fully understand how inheritance works or how to best leverage it. It will become clearer as we proceed on our journey.



The `ProxyUnicode` class should be treated as an implementation detail. The only important user-facing detail is that it allows PyNodes to have string methods on them (`.replace`, `.strip`, and so on). As of writing this book, I've never used the string methods on a `PyNode`. Maybe there are valid uses but I can't imagine them. There are always better, more explicit ways of dealing with the node. If you need to deal with its name, call a name-returning method (`name()`, `longName()`, and the like) and manipulate that. Use the `rename` method to rename the node.

Understanding PyMEL data and math types

PyMEL's intuitive use of type hierarchies does not end with Maya node types. It also provides a very useful wrapper around Maya's mathematical data types, including vectors, quaternions, and matrices. These types are located in the `pymel.core.datatypes` namespace.

Let's take a closer look at `xform`'s transform information.

```
>>> xform.translate
Attribute(u'pSphere1.translate')
>>> t = xform.translate.get()
>>> print t
[0.0, 0.0, 0.0]
```